Quick note for RSpec fans: unlike in RSpec, the name-munging in Minitest means that two tests in the same class cannot have the same string description—they'd resolve to the same Minitest name. Second quick note: there is a minitest-spec extension that gives Minitest a more RSpec-like syntax, but that version has been specifically turned down for usage in core Rails, and I won't be discussing it here.

Inside the test method you need to do two things. First, create a Task instance. Then, on lines 6 and 8, make your first assertions. Use refute to claim that task.complete? will be false on a new instance, and then after marking the task as complete, assert that the method call task.complete? will result in a true value. The assert method is the most basic of Minitest's assertions. It takes one argument, and the assertion passes if the argument is true (for Ruby values of true), and fails if the argument is false. The refute method is the exact opposite; it passes if the sole argument is false, and fails otherwise.

Minitest defines about a dozen assertion methods and their opposites. (See the documentation for a full list.[2]) The following table lists the six assertions you'll probably use most frequently.

Assertion	Passes if
assert(test)	test is true
assert_block block	The associated block returns true
assert_equal(expected, actual)	expected == actual
assert_includes(collection, object)	collection.include?(object)
assert_match(expected, actual)	expected =~ actual
assert_nil(expected)	expected == nil
assert_raises(exception) block	The associated block raises the exception

All Minitest assertions share two useful features. First, they all take an optional additional argument that is a string message. Here's an example:

```
assert_equal("Noel", author.name, "Incorrect author name")
```

If the assertion fails, the message is output to the console. I don't normally use message arguments, because the messages create more overhead and clutter the tests. But if you're in a situation where documentation is particularly important, messages can be useful to describe a test's intent.

Second, every assert method has an opposing refute method: refute_equal, refute_match, and so on. The refute methods pass where the assert methods would

2. http://docs.seattlerb.org/minitest/Minitest/Assertions.html